

多记忆的两阶段整理：先分别整理，再合并或排除

思路成立，但需要补一个「合并入口」

项目	睿乐大脑 · 整理模块
版本	V1（基于代码核查）
核查对象	整理任务执行结构
结论	Map 可做、Reduce 缺入口

3

可选实现路径

2

已接线可复用的关联

1

必须补的入口

多记忆的两阶段整理：先分别整理，再合并或排除

前置：多条记忆整理方案.md（记忆如何被选出来）、整理能力与10个模板匹配度评估.md（模板能力边界）

结论先行

1. 这个思路是对的，而且正好绕开单次整理的预算墙（18000 字符 / 单条 2200 / 上限 50 条）。分批 Map 之后，单次要处理的内容大幅下降，静默截断问题自然缓解。
2. 但现有代码不支持两阶段：整理任务是单次 LLM 调用直接出成果（`organize_workbench.go:416, 617`），没有父子任务、没有批次、没有多阶段编排（`ParentJobID` / `batch_id` 全仓零命中）。
3. 真正缺的不是 Map，是 Reduce。Map 用现有「一条记忆一个任务」就能拼出来；但整理任务的输入只能是记忆，不能是成果——「把多份成果合成一份」这个入口不存在。
4. 有一个零后端代码的绕法：把中间成果回写成「打了标记的记忆」，Reduce 就复用现有链路。代价是记忆列表被中间稿污染 + 有自我复读风险（下文有解）。
5. 一个本来就有、可直接变现的资产：成果 ↔ 来源记忆的关联表是真接线的（`organize_output_memories`，写入 `repository/organize.go:710-728`、读取 `:750-774`）。它让「覆盖率校验」和「去重判定」有据可依，而不是靠文本相似猜。

一、现状：整理任务的能力边界（已逐项核验）

能力	现状	证据
执行结构	单次 LLM 调用 → 一份成果，线性结束	<code>organize_workbench.go:416</code> 、 <code>:617</code>
阶段 (Stage)	只是展示用进度名： <code>loading_memories</code> (20) → <code>generating</code> (55) → <code>saving_output</code> (85) → <code>completed</code> (100)	<code>:407</code> 、 <code>606</code> 、 <code>626</code> 、 <code>434</code>
父子 / 批次任务	✗ 不存在	<code>ParentJobID</code> 、 <code>parent_job_id</code> 、 <code>batch_id</code> 全仓零命中
单次容量	上限 50 条；总预算 18000；单条截断 2200；先写后判 + 尾部静默截断	<code>:30-33</code> 、 <code>:943-952</code>
就绪校验	选中集合里任一条未就绪 → 整批失败	<code>organize.go:290-297</code>
失败降级	有本地兜底内容（ <code>fallback</code> 标记）	<code>:617</code> 、 <code>:431-432</code>
成果 ↔ 来源记忆	✅ 双向接线：写入 + 读取都真实存在	<code>repository/organize.go:710-728</code> 、 <code>:750-774</code>
按记忆反查成果	⚠️ 不对称：sprout 报告支持（ <code>JOIN organize_sprout_memories</code> ）， <code>outputs</code> 不支持	<code>repository/organize.go:636-639</code> ； <code>outputs</code> 侧无对应 <code>JOIN</code>
萌芽报告 (sprout)	✅ 单条记忆 → 异步简报，建报告时就写入记忆关联	<code>organize_sprout.go:29-69</code>
创建记忆	✅ 支持自定义 <code>metadata</code> 与 <code>source</code> （自由文本）	<code>types/organize.go:426-434</code>
<code>kind</code> 值域	⚠️ 有 <code>CHECK</code> 约束，只有 4 值： <code>note/record/audio/audio_card</code>	<code>migrations/versioned/000072_organize_section.up.sql:20-22</code>
记忆筛选	只支持 <code>kind</code> 精确 + <code>keyword</code> 模糊，没有排除式筛选	<code>repository/organize.go:160-163</code>

一句话：平台能做「一次性把一批记忆整理成一份成果」，做不了「整理结果再被整理」（no chaining）。这是两阶段方案的唯一硬缺口。

二、把你的思路拆成三段，逐段标"有什么 / 缺什么"

第 1 段：Map（分别整理）

选项	做法	评价
M1 一条一任务	每条记忆建一个整理任务 → N 份成果	零代码，但重：N 次 LLM 调用 + N 份完整 Markdown 成果
M2 一批一任务 (推荐)	按预算切成若干批（每批 ≤ 预算 80%，正常长度约 12—15 条）→ K 份成果	成本可控，批次即"分别整理"的单元
M3 用萌芽报告	每条记忆 → <code>POST /organize/sprout-reports/from-memory</code> 异步简报	已有异步 + 记忆关联，最轻，但简报不是为合并设计的结构化摘要

结论：Map 不缺能力，只缺"批次"这个概念。M2 用现有 API 就能实现（每批用一组 `memory_ids` 建任务）。

第 2 段：Reduce（合并）

这是缺口所在。

- 整理任务的输入只能来自 `organize_memories`（`executeOrganizeJob:602` 调 `ListMemoriesByIds`）。
- 成果（outputs）与萌芽报告（sprouts）都不能直接当输入。
- 所以 Reduce 必须先解决「中间产物怎么变回可整理的东西」。

第 3 段：排除

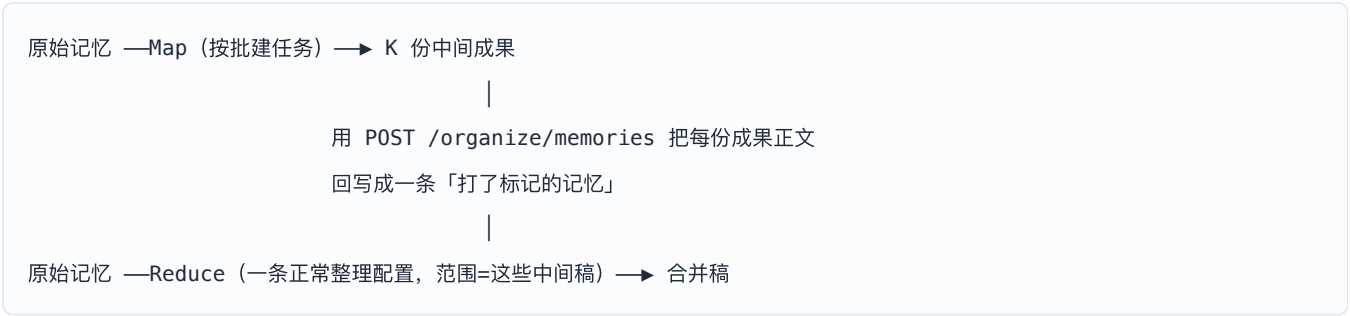
分两层，必须分开对待：

层级	含义	能力现状
排除输入	哪些记忆不参与本次合并	❌ 只有包含式筛选（kind / keyword），无排除式（如"排除已整理过的""排除中间稿"）
排除产出	合并时剔除重复 / 无关条目	💡 提示词能做，但必须显式列出被剔除项及原因，否则就是"AI 悄悄丢东西"

三、三条实现路径

方案 A | 回写法（零后端代码，建议先用来验证价值）

流程：



中间稿的标记方式（都靠已支持的字段）：

字段	值	用途
title	【中间稿】 <原主题>	肉眼可辨
source	整理中间稿	可被 keyword 命中，便于圈选/排查
metadata.stage	intermediate	机器可辨
metadata.origin_memory_ids	["mem_1","mem_2",...]	兜底保留原始引用链（见下方风险 3）
metadata.from_output_id	<成果 ID>	回溯

优点：全用现有 API，0 行后端代码，一两天内能跑通验证。

三个必须处理的风险：

- 自我复读 / 死循环（最严重） 中间稿本身就是一条记忆，会被下一次定时任务再次选中（定时任务取"最近 50 条"，而中间稿的 occurred_at 往往是最新的，会排在最前）。→ 缓解：Reduce 配置必须显式圈定中间稿范围；同时给 ListMemories 增加排除条件（如 source != '整理中间稿'）或 metadata 条件——这是小改动但不可省。
- 污染记忆列表 用户会在记忆列表里看到机器生成的中间稿。→ 缓解：kind 有 CHECK 约束不能新增值，所以只能靠 source / metadata 标记 + 前端过滤展示。这是方案 A 的固有代价。
- 引用链断裂（容易被忽略） 中间稿变成新记忆后，[M#] 的编号会重排，合并稿里的 [M#] 就再也回溯不到原始记忆。→ 缓解：中间稿的 metadata.origin_memory_ids 存原始 ID；Reduce 的提示词要求引用格式写成 【中间稿 ID:M#】，保留两级可追溯。

方案 B | 编排式（推荐作为正式版）

给「整理」补上多阶段能力：

改动	内容
表	<code>organize_jobs</code> 加 <code>parent_job_id</code> + <code>phase</code> (<code>map</code> / <code>reduce</code>)
编排	父任务按批派发 N 个 <code>map</code> 子任务；全部进入终态后，自动派发一个 <code>reduce</code> 任务
Reduce 输入	子成果的压缩摘要（每条结论一行 + 保留引用 + 来源子任务 ID）——不能喂全文，否则 Reduce 自己会撞同一个预算墙
失败容忍	子任务失败时用 <code>allow_partial</code> ，并在成果里标注「本批次缺失」
覆盖率	用 <code>organize_output_memories</code> 反查，成果上显示「本次覆盖 N / M 条记忆」

优点：干净、可追溯、不污染记忆、覆盖率可见。**成本**：1 个迁移 + service 编排 + 两套提示词。

注：给 `organize_outputs` 补一个「按记忆反查」的 JOIN（对齐 sprout 已有的做法）成本极低，性价比高——它是覆盖率校验的基础。

方案 C | 借用萌芽报告做 Map（混合）

Map 用 `sprout-reports/from-memory`（已异步、已带记忆关联、可按记忆反查），Reduce 仍需 A 或 B 的入口。
→ 只有 Reduce 侧先解决，C 才有意义；否则不如直接用 M2。

四、六个关键设计点（无论选哪条路都要先定）

1. **Map 粒度选「批」不选「条」**：每批 ≤ 预算的 80%（正常长度约 12—15 条）。按条虽然粒度细，但 N 次调用成本高，且同一件事跨条重复只能靠 Reduce 去重。
2. **Reduce 的输入必须是压缩摘要**，不是子成果全文。否则 Reduce 会重蹈覆辙撞预算墙（这是最容易踩的坑）。
3. **引用链必须两级可追溯**（原始记忆 ← 中间稿 ← 合并稿）。方案 A 若不存 `origin_memory_ids`，合并稿就是"无源之水"。
4. **排除必须可见**：合并成果末尾固定一节「已剔除：<内容> · 原因」。用户能接受"剔除了重复项"，不能接受"东西不见了"。
5. **失败要标注而不是静默**：某批失败 → 成果里写「第 3 批未参与合并」。这与平台「没依据写"记录中未提供"」的口径一致。
6. **幂等**：沿用现有 `dedupe_key` 防重机制（`organize_workbench.go:836`），避免重跑产生重复成果。

五、推荐路线

阶段	做什么	成本
第一步	用手工分批跑一遍：切 3 批 → 3 份成果 → 回写成中间稿 → Reduce 一次	0 行代码，验证合并质量与提示词
第二步	若质量达标：补 <code>ListMemories</code> 的排除条件（解决自我复读）+ 成果页「覆盖 N/M 条」提示	小
第三步	做方案 B 的编排（ <code>parent_job_id</code> + <code>phase</code> ），把两阶段产品化	中

验证时最该盯的：Reduce 出来的合并稿，有没有比"一次性把全部记忆喂进去"更好？如果差距不明显，说明两阶段的复杂度不值得——这一步的对照实验是决定要不要做 B 的唯一依据。

六、一句话口径

「先分别整理、再合并」在能力上成立，Map 现成、Reduce 要补：短期内用「成果回写成标记记忆」零代码跑通，长期靠给整理任务加父子阶段来产品化。排除建议放在 Reduce 的提示词里，但必须显式列出被剔除项。